Informe APE2_Parte 1

1. Datos Generales

Asignatura	Software Security
Ciclo	8 A
Unidad	1
Resultado de Aprendizaje de la Unidad	R1. Diseña e implementa software bajo los principios básicos de estándares seguros, en el marco de los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2 (Fases 1 a 4)
Título	Auditoría técnica y endurecimiento de la aplicación
Docente	Valeria Herrera Salazar
Integrantes	Dalton Maza, Leonardo Peralta, Gabriel Espinoza
Período	Del 26 de octubre al 27 de noviembre de 2026 (Semanas 5 a 8)
Horario	Jueves: 09h30 – 11h30 Viernes: 07h30 – 09h30
Lugar	Aula de Clase
Tiempo planificado	16 horas (4 horas por semana)

Tabla de sistematización de hallazgos criptográficos con su evidencia y clasificación preliminar de riesgo.

ID	Control evaluado	Evidencia	Resultado	Riesgo preliminar
C1	Hash de contraseñas en reposo	hashPassword.js (bcrypt); colección accounts con 2*b*10$...	Cumple	Bajo
C2	Contraseñas SMTP en texto plano	authController.js (transporter con user/pass literales)	No cumple	Crítico
C3	Credenciales WiFi/MQTT en el firmware	Pis_6to.ino (ssid, password, mqtt_user, mqtt_password)	No cumple	Crítico
C4	TLS en tránsito MQTT	Puerto 1883, openssl s_client falla; PubSubClient sin cifrar	No cumple	Crítico
C5	TLS/HTTPS en front y API	localhost:3000 en HTTP; sin cabeceras de seguridad	No cumple	Alto

C6	Token de recuperación en reposo	Campo accounts.token en claro (texto plano en Mongo)	Parcial	Alto
C7	Gestión de sesiones JWT	Firma verificada (lab escenario 5); sin exp corto ni revocación	Parcial	Alto
C8	Política de contraseñas	Mínimo 5 caracteres en modelo; sin complejidad	Parcial	Alto
C9	Dependencias criptográficas	crypto@1.0.1 obsoleto en package.json	No cumple	Alto
C10	protección anti fuerza bruta	bcrypt lento pero sin rate limiting en /auth	Parcial	Alto

Sistematización de los hallazgos criptográficos con su evidencia y clasificación preliminar de riesgo.

Hallazgo	**Evidencia**	**Riesgo preliminar**	**acción correctiva**
C2 — Credencial de correo institucional en el código fuente	authController.js, líneas del transporter nodemailer	Critico	Rotar contraseña del correo hoy; mover a .env; purgar historial Git (git filter-repo / BFG)
C3 — Secretos WiFi/MQTT en el sketch ESP32	Pis_6to.ino constantes ssid, password, mqtt_user, mqtt_password (las antiguas comentadas)	Critico	Rotar credenciales del broker y WiFi; usar WiFiManager o variables de compilación; purgar repo
C4 — Tráfico MQTT en texto plano	client.setServer(mqtt_server, 1883); prueba openssl s_client/mosquitto_sub	Critico	Habilitar TLS 8883 en el broker y WiFiClientSecure + huella digital en el ESP32
C5 — Sin HTTPS ni cabeceras de seguridad	curl -sI localhost:3000 sin HSTS/CSP/X-Frame-Options; app.js sin Helmet	Alto	Desplegar con TLS (Vercel/Proxy inverso) y
C6 — Token de recuperación sin cifrar en BD	Modelo Account.token en claro	Alto	Hashear el token al guardarlo (comparar hash al usar) o cifrar a nivel de campo
C7/C8/C10 — Sesión y política de contraseñas	JWT sin expiración corta/revocación; minlength:5; sin rate limit	Medio-Alto	exp de 15 min + refresh; rate limit en /auth; fortaleza mínima en servidor
C9 — Dependencia criptográfica obsoleta	package.json: crypto@^1.0.1	Alto	Eliminar paquete (Node ya incluye crypto nativo); npm audit fix

Revisión de los criterios de evaluación: fortaleza del algoritmo, uso de salt por usuario, gestión de los secretos de configuración y separación de entornos.

- **Fortaleza del algoritmo:** Las contraseñas de usuario cumplen al emplear bcrypt en hashPassword.js, pero se requiere eliminar el paquete obsoleto crypto@^1.0.1 en package.json para usar el módulo nativo de Node.js, además de habilitar TLS en el puerto 8883 para MQTT y HTTPS en la API.
- **Uso de salt por usuario:** bcrypt genera automáticamente un salt único por registro ($2b$10$... en la colección accounts), protegiendo contra tablas rainbow. Este principio debe extenderse al token de recuperación (Account.token), que actualmente se almacena en texto plano.
- **Gestión de secretos de configuración:** No cumple; existen credenciales SMTP en texto plano en authController.js y credenciales WiFi/MQTT expuestas en Pis 6to.ino.
- **Separación de entornos:** No cumple; la API corre en desarrollo sobre HTTP (localhost:3000) sin cabeceras de seguridad ni separación mediante variables de entorno (.env).

Búsqueda de credenciales y secretos fijos en el código mediante revisión manual y búsqueda de patrones; redacción de la propuesta de migración del almacenamiento de contraseñas y del plan de gestión de secretos del proyecto.

Búsqueda de credenciales y secretos expuestos

```
leoma@LaPoderosa:~/octavo/security/Security8tavo/code/back-end_PIS6TO$ grep -rnE "(password|pass|mqtt_user|token)\s*[:]"
 . --exclude-dir=node_modules
./src/models/Account.js:25:    password: {
./src/models/Account.js:37:    token: {
./src/validationSchemas/login.js:7:  password: Joi.string().required().min(5).max(61).messages({
./src/validationSchemas/account.js:14:  password: Joi.string().required().min(5).max(30).messages({
./src/validationSchemas/account.js:39:  password: Joi.string().optional().min(8).alphanum().max(30).messages({
./src/controllers/authController.js:16:        pass: 'tedquieromucho' // Contraseña del servidor
./src/middlewares/mqtt.js:14:  password: process.env.MQTT_PASSWORD || undefined,
./scripts/seedUsers.js:35:    password: await hashPassword(password),
./scripts/seedUsers.js:50:      password: "admin123",
./scripts/seedUsers.js:58:      password: "user123",
```

Se encontró como exposición de secretos en seedUsers y authController.

seedUsers fue un archivo de población de la base de datos, que está definido dentro del .gitignore para no subirlo al repositorio del código por accidente.

Propuesta de migración criptográfica

- **Hashing de tokens de recuperación (C6):** Reemplazar el almacenamiento en claro de Account.token generando un token aleatorio con crypto.randomBytes(32).toString('hex') y guardando únicamente su hash SHA-256 en MongoDB para compararlo al validar el restablecimiento.
- **Dependencias y política de acceso (C7, C8, C9, C10):** Desinstalar crypto@^1.0.1 (npm uninstall crypto); elevar minlength de 5 a 10 caracteres exigiendo complejidad en el esquema de Mongoose; configurar expiración de JWT a 15m con refresh tokens; e instalar express-rate-limit en /auth.
- **Cifrado en tránsito (C4, C5):** Activar helmet() en app.js y desplegar tras proxy inverso con TLS/HTTPS; en el ESP32 (Pis 6to.ino), migrar de PubSubClient en puerto 1883 a WiFiClientSecure sobre el puerto 8883 validando el certificado del broker.

Plan de gestión de secretos

- **Rotación inmediata:** Cambiar la contraseña del correo institucional usado en nodemailer y las credenciales del broker MQTT y red WiFi
- **Externalización en Backend (C2):** Mover usuario y contraseña SMTP a un archivo .env, añadir .env al .gitignore y versionar únicamente una plantilla .env.example con claves vacías.
- **Externalización en Firmware ESP32 (C3):** Extraer ssid, password, mqtt_user y mqtt_password de Pis 6to.ino hacia un archivo secrets.h excluido en .gitignore (o implementar el portal cautivo WiFiManager).
- **Saneamiento de Git:** Eliminar los secretos del historial de commits mediante git filter-repo o BFG Repo-Cleaner y forzar el push al repositorio remoto.

6. Bibliografía:

- Haz Lopez, L. V., et al. (2024). Hacking Ética: Teoría y Prácticas. CID. ISBN: 978-1967344-23-9.
- Vega Briceño, E. (2021). Seguridad de la Información. Editorial Área de Innovación y Desarrollo. ISBN: 978-84-122093-6-5.
- Whitman, M. E., & Mattord, H. J. (2018). Management of Information Security (6th ed.). Cengage Learning.
- Stallings, W. (2020). Cryptography and Network Security: Principles and Practice (8th ed.). Pearson.
- OWASP Foundation. (2025). OWASP Cheat Sheet Series: Password Storage. Recuperado de https://cheatsheetseries.owasp.org/